Les bases de JavaScript

Préliminaires

Pour exécuter du code JavaScript, il faut :

- créer un fichier .js (par exemple script.js)
- ajouter dans le head du fichier html une balise script indiquant le chemin du fichier JS :
  `<script src="script.js"></script>`

Il est aussi possible de mettre du code JS directement dans le fichier HTML dans une balise `<script></script>` , mais cela n'est pas une bonne pratique pour la lisibilité et la maintenance.

Partie 1 : Les variables avec JavaScript

I) Les variables de type « simple »

1) Déclarez une variable en JavaScript

```
let monAge = 42
```

On peut faire évoluer cette valeur en écrivant : `monAge = 43`

On peut aussi déclarer des constantes qui ne peuvent plus être changées par la suite (sinon JS renvoie une erreur) :

```
const monPrenom = "David"
```

Remarque :

- mettre « `;` » pour indiquer la fin d'une ligne de code n'est pas obligatoire en JS.
- var peut être utilisée pour déclarer une variable, mais elle est aujourd'hui obsolète.

Débugueur son code avec console.log()

Pour vérifier le contenu d'une variable, il est possible d'utiliser l'instruction *console.log()*

```
let monAge = 42
console.log(monAge)
```

le résultat de console.log() se lit dans le navigateur :

- Inspecter (ou F12)
- onglet « console »

Les types de données basiques

- String : correspond à une chaîne de caractères, c'est-à-dire un texte
- number : correspond à un nombre
- boolean : correspond à un booléen (il n'accepte que deux valeurs : true ou false)

2) Utilisez des opérateurs

Manipulez des chiffres

```
const entreesPremiereSemaine = 1000
const entreesDeuxiemeSemaine = 2000
const totalEntrees = entreesPremiereSemaine + entreesDeuxiemeSemaine

let placesDejaVendues = 150
placesDejaVendues = placesDejaVendues + 10
```

ou

```
let placesDejaVendues = 150
placesDejaVendues += 10
```

On peut utiliser les opérateur +, -, /, * ou `+=, -=, *=, /=`.

Manipulez des chaînes de caractères avec la concaténation

```
let premierePartie = "Mon nom est Bond"
let deuxiemePartie = ", James Bond."
let punchline = premierePartie + deuxiemePartie
// punchline vaut "Mon nom est Bond, James Bond."
```

ou

```
let punchline = "Mon nom est Bond"
punchline += ", James Bond."
```

Remarque :

- Les caractères « `//` » permettent d'insérer des commentaires dans le code.
- Différencier les types de données

```
let placesDejaVendues = "150"
placesDejaVendues += "10"
console.log(placesDejaVendues)
// affichera "15010" et pas "160"
```

On résout le problème avec la fonction Number() :

```
let placesDejaVendues = Number("150")
placesDejaVendues += Number("10")
console.log(placesDejaVendues)
```

II) Les variables de type Object

Un **Object** JavaScript est un **conteneur** composé de **propriétés** qui ont chacune une valeur.
On peut ainsi stocker plusieurs valeurs en même temps

1) Déclarez un objet en JavaScript

Pour déclarer un objet en JavaScript, on utilise les accolades `{ }` .

```
let personnage = {
nom: "Wayne",
prenom: "Bruce",
age: 35,
}
```

l'objet personnage à 3 propriétés (nom, prenom et age) qui possèdent chacune une valeur.

2) Manipuler les objets

Ajoutez (ou remplacez, si elle existe déjà) une propriété à un objet JavaScript

```
personnage.couleurPrefere = "rouge"
```

Accédez à une propriété d'un objet JavaScript

```
console.log(personnage.nom)
// affichera "Wayne"
```

3) Regroupez des données grâce aux tableaux

Un **tableau** en JavaScript est un **objet** qui permet de lister plusieurs variables ou valeurs, et de les regrouper.

Déclarez un tableau en JavaScript

```
const maCollectionDeFilms = ["Titanic", "Jurassic Park"]
```

On peut stocker des variables dans un tableau :

```
const monFilmPrefere = "Titanic"
const monPremierFilm = "Jurassic Park"
const maCollectionDeFilms = [monFilmPrefere, monPremierFilm]
// maCollectionDeFilms vaut ["Titanic", "Jurassic Park"]
```

Accédez à un élément du tableau

```
let premierFilmDuTableau = maCollectionDeFilms[0]
```

0 représente le premier élément du tableau, 1 le second, etc.

Comptez le nombre d'éléments de votre tableau

```
const nombreDeFilms = maCollectionDeFilms.length
console.log(nombreDeFilms)
// nombreDeFilms vaut 2
```

Modifiez votre tableau grâce aux méthodes

- Ajoutez des données grâce à la méthode push

```
let mesFilms = ["Titanic", "Jurassic Park"]
mesFilms.push("Retour vers le futur")
console.log(mesFilms)
// mesFilms vaut ["Titanic", "Jurassic Park", "Retour vers le futur"]
```

- Supprimez des données grâce à la méthode pop

```
let mesFilms = ["Titanic", "Jurassic Park", "Retour vers le futur"]
mesFilms.pop()
// mesFilms vaut ["Titanic", "Jurassic Park"]
```

III) Distinguez les copies par "valeur" et par "référence"

La copie par valeur

```
let variableSimple1 = 25
let variableSimple2 = variableSimple1
```

Si on change la valeur de variableSimple1, alors variableSimple1 et variableSimple2 auront des valeurs différentes.

La copie par référence

```
let variableObjet1 = ['pomme', 'cerise']
let variableObjet2 = variableObjet1
```

Ici, si on change la valeur de variableObjet1, alors variableObjet2 sera également modifiée.

Dans la copie par valeur (pour les variables de type simple), la mémoire dans laquelle est stockée la variable est dupliquée. Alors que dans la copie par référence, c'est une étiquette pointant vers le même emplacement mémoire qui a été dupliqué. Quand on change donc la valeur du contenu de la mémoire en passant par une étiquette ou par une autre, c'est la valeur des deux étiquettes qui est modifiée.

Remarque : les tableaux sont souvent déclarés avec *const* en JavaScript, même s'ils évoluent tout au long du code. Ce qui est constant, ce n'est pas le contenu, mais l'**endroit en mémoire où est stocké le tableau**.

```
const fruits = ["pomme", "banane"]
fruits.push("cerise") // ok
fruits = ["kiwi"] // ❌ erreur
```

Partie 2 : L'algorithmie avec JavaScript

I) La logique de programmation

Un **algorithme** est une façon de concevoir les étapes d'un code dans le but de **résoudre un problème donné**.
Les **blocs de code** sont des regroupements de lignes de code associés à des **structures conditionnelles**, des **boucles** et à des **fonctions**. Ils permettent d'organiser le code et de clarifier à quoi sert un groupe de lignes de code. En JavaScript, ils sont délimités par des accolades `{ }` .

```
{
    const monChiffre = 4
    console.log(monChiffre)
}
```

II) Contrôlez du code grâce aux conditions

1) La conditions if / else pour gérer une seule réponse

```
if (condition) {
    // Code exécuté si la condition est vraie
} else {
    // Code exécuté si la condition est fausse
}
```

La condition peut être :

- Un booléen
- une comparaison

```
let estMajeur = true
if (estMajeur) {
    console.log("Vous êtes majeur")
} else {
    console.log("Vous êtes mineur")
}
```

```
let age = 20
if (age >= 18) {
    console.log("Vous êtes majeur")
} else {
    console.log("Vous êtes mineur")
}
```

2) Les conditions if/else if/else et switch/case pour gérer plusieurs réponses

Lorsque l'on doit gérer plusieurs réponses, on peut utiliser if/else if/else ou switch.

Structure conditionnelle *if/else if/else*.

```
if (age >= 65) {
    console.log("Vous êtes sénior")
} else if (age >= 18) {
    console.log("Vous êtes majeur")
} else {
    console.log("Vous êtes mineur")
}
```

Structure conditionnelle *switch/case* *(*switch **:** *valeur à tester, case : valeurs possibles)*

```
let note = "A"
switch (note) {
    case "A":
        console.log("Excellent travail !")
        break
    case "B":
        console.log("Bon travail !")
        break
    case "C":
        console.log("Travail correct !")
        break
    case "D":
        console.log("Insuffisant !")
        break
    default:
        console.log("Note invalide")
}
```

Break sert à quitter le switch après être entré dans une condition. Il est impératif de ne pas l'oublier.

II) Répétez du code grâce aux boucles

1) La boucle for

Elle est utilisée lorsque l'on sait d'avance combien de fois il faudra répéter le code dans la boucle.

```
for (let compteur = 0; compteur < 3; compteur = compteur + 1) {
    console.log(compteur)
}
```

2) La boucle while

Elle est utilisée pour répéter du code autant de fois qu'il le faut pour qu'une condition ne soit plus vraie.

```
let motDePasse = "Bon travail !"
let bonMotDePasse = "toto123"
while (motDePasse !== bonMotDePasse) {
     motDePasse = prompt("Entrez votre mot de passe.")
}
console.log("Mot de passe correct !")
```

3) La boucle forEach

Elle est utilisée pour parcourir chaque élément d'un tableau

```
let etudiants = ["Alice", "Karim", "Sofia", "Lucas"]
etudiants.forEach(function(etudiant) {
      console.log("Bonjour " + etudiant)
})
```

IV) Organisez votre code

1) Utiliser les fonctions

Une **fonction** est un bloc de code que l'on peut appeler pour exécuter le code qu'elle contient.
Une fonction peut :

- effectuer une action
- retourner quelque chose

Dans les deux cas, elle peut prendre en entrée 0, 1 ou plusieurs paramètres pour déterminer son comportement.

Exemple :

```
function saluer(nom) {
    console.log('Hello ' +nom)
}
saluer('Jean')
// Affichera : Hello Jean
```

```
function additionner(a, b) {
    return a + b
}
console.log(additionner(5, 3))
// Affichera : 8
```

Les fonctions fléchées

Les fonctions fléchées ont été introduites pour simplifier la syntaxe des fonctions et améliorer la manipulation des objets.

```
const saluer = (nom) => {
    console.log('Hello ' + nom)
}
saluer('Jean')
```

```
const additionner = (a, b) => {
    return a + b
}
console.log(additionner(5, 3))
```

Les deux notations sont équivalentes, c'est juste la syntaxe qui change.

Remarques :
S'il n'y a qu'un seul paramètre, on peut se passer des parenthèses et s'il n'y a qu'une seule instruction dans la fonction on peut se passer des accolades pour écrire une fonction fléchée courte.

```
const saluer = (nom) => {
    console.log('Hello ' + nom)
}
```

Peut s'écrire :

```
const saluer = nom => console.log('Hello ' +nom)
```

Dans les fonction fléchée courte, on ne note pas le return :

```
const additionner = (a, b) => {
    return a + b
}
```

S'écrit :

```
const additionner = (a, b) => a + b
```

Les fonctions anonymes

Il existe aussi des fonctions dites anonymes, car elle n'ont pas de nom. Nous en avons vu une dans le chapitre sur le forEach :

```
let etudiants = ["Alice", "Karim", "Sofia", "Lucas"]
etudiants.forEach(function(etudiant) {
      console.log("Bonjour " + etudiant)
})
```

Les fonctions anonymes peuvent aussi être fléchées (courte ou non) :

```
let etudiants = ["Alice", "Karim", "Sofia", "Lucas"]
etudiants.forEach((etudiant) => {
     console.log("Bonjour " + etudiant)
})
```

ou

```
etudiants.forEach((etudiant) => console.log("Bonjour " + etudiant))
```

2) Codez proprement

Les fonctions sont un élément majeur du code. Il est donc très important de **respecter quelques bonnes pratiques** :

- Définir une seule tâche spécifique pour chaque fonction (afficher, calculer, lancer une opération, etc.)
- Écrivez des fonctions courtes (30 lignes max) : c'est une conséquence directe du point précédent
- Nommez clairement les fonctions et leurs paramètres (ceci est valable également pour les variables)

Découper le code en plusieurs fonctions permet de mieux l'organiser, mais il est aussi très utile de séparer le code dans plusieurs fichiers que l'on pourra appeler avec plusieurs balises script :

```
<script src="config.js"></script>
<script src="script.js"></script>
```

3) Maîtrisez la portée des variables

Les variables ont une **portée**, qui est définie par l'endroit où cette variable est déclarée. Ainsi, on distingue deux types de variables :

- **les variables globales** : visibles et utilisables dans l'ensemble du code
- **les variables locales** : utilisables qu'au sein du bloc de code { } dans lequel elles sont définies.

Exemple :

```
let monNombre = 1     // variable globale

function afficheUnNombre() {
    let monNombreLocal = 2       // variable locale
    console.log(monNombre) // monNombre est accessible
    console.log(monNombreLocal) // monNombreLocal est accessible
}

console.log(monNombre)
Affichera : 1. Car monNombre est accessible.
console.log(monNombreLocal)
Affichera : ReferenceError: inconnue is not defined.
Car monNombreLocal n'est pas accessible en dehors du bloc de code de la
fonction.
afficheUnNombre()
Affichera : 1 et 2. Car monNombre et monNombreLocal  ne sont pas accessibles.
```

4) Les modules JavaScript : import / export

Les **modules** permettent de **séparer le code** en plusieurs fichiers et de **réutiliser des fonctions, objets ou classes** facilement.

Pour cela, il faut déclarer les fonctions (par exemple) comme exportables avec `export` :
Dans math.js

```
export const additionner = (a, b) => a + b;
export const multiplier = (a, b) => a * b;
```

Il suffit ensuite de les importer dans un autre fichier avec `import`, en spécifiant le chemin du fichier
Dans app.js

```
import { additionner, multiplier } from './math.js';
console.log(additionner(5, 3)); // 8
console.log(multiplier(5, 3));  // 15
```

Partie 3 : Faites interagir JavaScript avec une page HTML

I) Manipuler les éléments d'une page web

JavaScript permet de manipuler directement une page HTML pour la rendre **interactive**.

1) La structure d'une page web : DOM

Généralement, une page web est constituée de deux parties :

- Un **head** qui donne des informations générales sur la page (title, meta, link, style, script)
- un **body** qui contient ce qui est visible sur la page

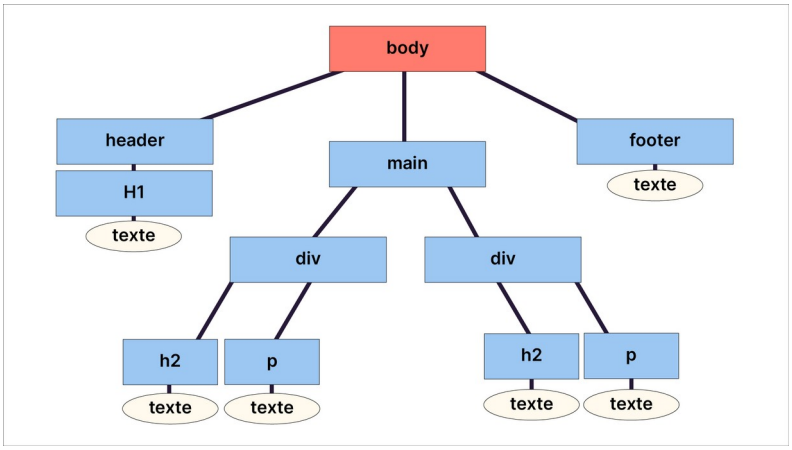
Voici un exemple de body pour une page simple :

```
<body>
    <header>
        <h1>AzerType</h1>
    </header>

    <main>
        <div>
            <h2>Le jeu</h2>
            <p>Un petit texte</p>
        </div>
        <div>
            <h2>Une autre div</h2>
            <p>Un autre texte</p>
        </div>
    </main>

    <footer>
        @Copyright : OpenClassrooms.
    </footer>
</body>
```

Ce **body** est constitué d'un **header** avec un titre, d'un corps (**main**) et d'un **footer**.
JavaScript voit une page HTML comme une structure de nœuds représentant les balises, organisés en relation parent/enfant, appelée le DOM (Document Object Model).



La structure du DOM ci-dessus peut être représentée comme un arbre :

- le <body> est un **nœud** du DOM
- il mène à des **nœuds enfants**: <header>, *<main>*, *<footer>*
- ces enfants peuvent eux-mêmes avoir des nœuds enfants : <h1> pour <header>, 2 pour main, aucun pour <footer>
- chaque « branche » se termine généralement par un nœud texte.

Pour JavaScript, chaque balise HTML dans le DOM (body, main, h1, div, etc.) est représenté par un **objet HTMLElement** qui possède :

- des propriétés : id, class, contenu, etc.
- des méthodes qui permettent d'interagir avec l'élément : réagir à un événement, modifier le contenu ou le style, etc.

2) Récupérez un élément du DOM

Utiliser defer pour différer l'exécution du script

Pour manipuler le DOM, JavaScript utilise une variable globale, ***document***, qui représente la page HTML entière. La page HTML doit donc être entièrement chargée pour que le code JavaScript puisse s'exécuter correctement. Or, le script étant lancé dans la balise head, avant le body, la page HTML n'est pas encore chargée entièrement lorsque le script JS est appelé.

Pour résoudre ce problème, on ajoute defer à la balise script pour demander au navigateur de jouer le script JS une fois seulement la page HTML chargée.

```
<script src="script.js" defer></script>
```

Différentes syntaxes pour récupérer un élément

JavaScript propose beaucoup de méthodes pour récupérer les éléments du DOM, dont voici quelques exemples :

- getElementById()
- querySelector()
- querySelectorAll()
- getElementsByClassName()

getElementById : permet de récupérer un élément en fournissant son id en paramètre.

```
<body>
    <div id="voiture">Peugeot</div>
</body>
```

Pour accéder à cette balise, on écrit :

```
let baliseVoiture = document.getElementById("voiture")
console.log(baliseVoiture)
```

Le console.log montre qu'on a bien récupéré notre objet HTMLElement : `▶ <div id="zoneProposition">`

Si on clique sur le petit triangle à côté de cette div pour déployer le contenu, les détails de cet objet HTMLElement s'affichent :

```
▶ attributes: NamedNodeMap [ id="voiture" ]
  autocapitalize: ""
  autocorrect: true
  autofocus: false
  baseURI: "http://localhost:8080/"
  childElementCount: 0
▶ childNodes: NodeList [ #text ]
▶ children: HTMLCollection { length: 0 }
▶ classList: DOMTokenList []
  className: ""
  clientHeight: 10
  clientLeft: 0
  clientTop: 0
  clientWidth: 1904
  contentEditable: "inherit"
  currentCSSZoom: 1
▶ dataset: DOMStringMap(0)
  dir: ""
  draggable: false
  enterKeyHint: ""
▶ firstChild: #text "Peugeot"
```

Sur les objets HTMLElement, JavaScript propose beaucoup de propriétés et de méthodes qui sont autant d'outils utilisables en fonction de vos besoins.

HTLMElement étant un objet, on peut accéder aux propriétés du nœud (par exemple la hauteur de l'élément) :

```
console.log(baliseVoiture.clientHeight)
```

querySelector : permet de récupérer un élément en fournissant son sélecteur CSS en paramètre

```
<body>
    <div id="voiture"><span>Cachalot</span></div>
</body>
```

Pour mettre le mot *Peugeot* en gras en CSS, nous aurions écrit :

```
#voiture span {
    font-weight: bold;
}
```

querySelector nous permet de trouver le premier élément qui correspond au sélecteur CSS proposé :

```
let baliseSpanVoiture = document.querySelector("#voiture span")
console.log(baliseSpanVoiture)
```

```
▼ <span>
    accessKey: ""
    accessKeyLabel: ""
```

querySelectorAll : le principe est le même que pour tous les éléments dans une liste de nœuds (*NodeList*).

```
<body>
    <div class="choixCouleur">
        <input type="radio" name="couleur" id="rouge" value="1" checked>
        <label for="rouge">rouge</label>
        <input type="radio" name="couleur" id="bleu" value="2">
        <label for="bleu">bleu</label>
    </div>
</body>
```

La div contient deux inputs de type radio qu'on peut récupérer en une seule fois :

```
let listeInputRadio = document.querySelectorAll(".choixCouleur input")
console.log(listeInputRadio)
```

```
▶ NodeList [ input#rouge , input#bleu ]
```

Si on clique sur le triangle, on voit la NodeList :

```
▼ NodeList [ input#rouge , input#bleu ]
  ▶ 0: <input id="rouge" type="radio" name="couleur" value="1" checked=""> 
  ▶ 1: <input id="bleu" type="radio" name="couleur" value="2"> 
    length: 2
  ▶ <prototype>: NodeListPrototype { item: item(), keys: keys(), values: values(), … }
```

On peut ensuite cliquer sur le triangle de chaque input pour en voir les propriétés.
On peut utiliser une boucle pour parcourir les différents éléments de cette liste un par un pour y accéder.

```
for (let i = 0; i < listeInputRadio.length; i++) {
    console.log(listeInputRadio[i])
}
```

getElementsByClassName : le principe est le même que pour tous les éléments dans une liste de nœuds (*NodeList*).

```
<body>
    <div class="produit">Produit A</div>
    <div class="produit">Produit B</div>
    <div class="produit">Produit C</div>
</body>
```

La div contient trois div avec la classe produit qu'on peut récupérer en une seule fois :

```
let listeProduit = document.getElementsByClassName("produit")
console.log(listeProduit)
```

```
▶ HTMLCollection { 0: div.produit , 1: div.produit , 2: div.produit , length: 3 }
```

II) Modifiez un élément d'une page web

1) Modifiez un élément du DOM

Une balise se définit par son nom et ses attributs.

```
<img id="premiereImage" src="image.jpg" alt="Ceci est une image de test">
```

Cette balise possède :

- un nom : *img*
- 4 attributs : id, src, alt, class

JavaScript propose beaucoup de méthodes et propriétés pour modifier les éléments du DOM, dont voici quelques exemples :

- setAttribute()
- removeAttribute()

- classList
- innerHTML
- textContent

<u>setAttribute :</u> permet de créer ou modifier un attribut

```
<img id="premiereImage" src="image.jpg" alt="Ceci est une image de test">
```

Pour modifier un attribut

```
let baliseImage = document.getElementById("premiereImage")
baliseImage.setAttribute("alt", "Ceci est une photo de profil")
```

Dans ce code :

- On récupère la balise img grâce à son id
- On change la valeur de l'attribut alt pour lui donner la valeur « Ceci est une photo de profil »

setAttribute est générique et s'appliquer à tous les attributs. Pour les attributs valides en HTML de base, il n'est pas nécessaire d'utiliser la méthode *setAttribute*. On peut faire plus simple :

```
baliseImage.alt = "Ceci est une photo de profil"
```

Pour créer un attribut :

```
baliseImage.setAttribute("class", "photo")
```

<u>removeAttribute :</u> permet de supprimer un attribut

```
let baliseImage = document.getElementById("premiereImage")
baliseImage.removeAttribute("alt")
```

L'attribut alt de l'image est supprimé et l'élément n'a plus de texte alternatif

<u>classList :</u> permet de modifier une classe

```
<div class="listeMots centree actif photo"></div>
```

On peut modifier une classe avec setAttribute(), mais ce n'est pas pratique car il faut réécrire toutes les classes. ClassList simplifie la procédure.

```
baliseImage.classList.add("nouvelleClasse")
baliseImage.classList.remove("photo")
```

la balise devient alors :

```
<div class="listeMots centree actif nouvelleClasse"></div>
```

<u>innerHTML :</u> récupère ou remplace tout le contenu HTML, y compris les balises.

```
<div id="monDiv">
     <p>Bonjour !</p>
</div>
```

Pour récupérer le contenu HTML :

```
let contenu = document.getElementById("monDiv").innerHTML
console.log(contenu)       // Affichera : <p>Bonjour !</p>
```

Pour remplacer le contenu HTML :

```
let contenu = document.getElementById("monDiv")
contenu.innerHTML = "<h1>Bonjour !</h1>"
console.log(contenu.innerHTML)       // Affichera : <h1>Bonjour !</h1>
```

<u>textContent</u> : permet de récupérer ou de modifier le texte d'un élément

```
<div id="monDiv">
     <p>Bonjour !</p>
</div>
```

Pour récupérer le texte :

```
let texte = document.getElementById("monDiv").textContent
console.log(texte)         // Affichera : Bonjour !
```

Pour remplacer le texte :

```
let texte = document.getElementById("monDiv")
texte.textContent = "Hello !"
console.log(texte.textContent )      // Affichera : Hello !
```

2) Utilisez l'inspecteur pour vérifier les modifications

Pour vérifier les modifications sur un élément particulier, il faut :

- Dans le navigateur placer la souris sur cet élément
- Faire un clic droit, puis « inspecter »
- Vérifier dans la console que notre élément a bien été modifié.

III) Créez un nouvel élément dans une page web

Pour créer des éléments, JavaScript nous propose plusieurs manières de faire. Voici deux exemples :

- La méthode createElement()
- La propriété innerHTML

1) Créer un nouvel élément avec la méthode createElement

createElement : permet de créer n'importe quelle balise

```
let nouvelElement = document.createElement("h1")
```

Une fois la balise créée, on peut la configurer avec setAttribute() et textContent.

```
nouvelElement.textContent = "je suis un nouvel élément"
```

Insérez la balise créé dans la page :

Une fois l'élément créé, il faut l'insérer dans le DOM pour qu'il s'affiche sur la page.
Pour cela, il faut :

- Déterminer quel sera l'élément parent
- *Utiliser appendChild* ("ajouter un enfant")

```
let parentElement = document.getElementById("conteneur")
parentElement.appendChild(nouvelElement)
```

Le code HTML rendu sera :

```
<div id="conteneur">
     <h1> je suis un nouvel élément</h1>
</div>
```

Il existe d'autres méthodes pour insérer des éléments dans le DOM :

- prepend : ajouter au début
- before : ajouter avant
- insertAdjacentElement: ajouter à un endroit précis

2) Créer plusieurs éléments avec la propriété innerHTML

La méthode createElement devient difficile à utiliser lorsqu'on veut créer ou imbriquer de nombreux éléments. Pour faire cela, on utilise **l'interpolation.**

L'interpolation consiste à entourer une chaîne de caractères avec des backticks : ` (altrGr + 7). Cela permet de d'insérer des variables avec ${ } et de tenir compte des sauts de lignes et de la tabulation.

```
let contenuTitre = "Azertype"
let contenuParagraphe = "L'application pour apprendre à taper plus vite !"
```

```
let div = `
    <div>
        <h1>${contenuTitre}</h1>
        <p>${contenuParagraphe}</p>
    </div>
    `
```

Insérez le code HTML créée dans la page :

```
let body = document.querySelector("body")
body.innerHTML = div
```

IV) Interagissez avec un élément d'une page web grâce aux événements

1) La programmation événementielle

Jusqu'à présent, le code s'est toujours exécuté de manière **séquentielle** : les instructions s'exécutent les unes après les autres. On peut aborder une nouvelle manière d'envisager la programmation, avec la **programmation événementielle**.

Un **événement** correspond à une action spécifique : clic sur un bouton, frappe d'une touche du clavier, etc.
La programmation événementielle consiste à exécuter du code au moment où un événement donné se produit.

En pratique, il faut :

- Demander à JavaScript d'écouter l'événement qui nous intéresse avec un **eventListener**
- Puis lier l'événement à un bloc de code

2) Écouter des événement avec addEventListener

addEventListener() est une méthode qui peut être appelée directement depuis un élément HTML. Elle prend deux paramètres :

- Le nom de l'événement (click, keypress, submit, input, etc.)
- Une fonction qui est appelée lorsque l'événement se déclenche

Dans le fichier HTML :

```
<button id="monBouton">Cliquez-moi !</button>
```

Dans le fichier JavaScript :

```
let monBouton = document.getElementById("monBouton")
monBouton.addEventListener("click", function () {
        console.log("Vous avez cliqué sur le bouton")
    })
```

ou avec une fonction fléchée :

```
monBouton.addEventListener("click", () => {
    console.log("Vous avez cliqué sur le bouton")
})
```

Dans le fichier JS :

- on récupère le bouton
- on lui ajoute un écouteur avec addEventListener à qui on passe :
    - un événement à écouter
    - une fonction à exécuter lorsque l'événement se produit

Cette fonction sera appelée autant de fois que l'événement se produit

2) Récupérez les informations sur un événement avec la variable “event”

On peut avoir besoin des informations sur l’événement qui vient de se dérouler. Par exemple :

- Quel élément l’utilisateur a-t-il cliqué ?
- Quelle touche du clavier l’utilisateur a-t-il appuyé ?
- Quelles étaient les coordonnées de la souris

```
document.addEventListener('keydown', (event) => {
    if (event.key === "a") {
        console.log("La touche A a été pressée")
    }
})
```

event est un objet JS qui contient toutes les informations liées à l’événement.
Il possède de nombreuses propriétés et méthodes utiles, comme par exemple :

- event.target : renvoie l’élément HTML qui a déclenché l’événement
- event.type : renvoie le type d’événement (click, keydown, etc.)
- event.key : touche pressée pour les évènement lié au clavier
- event.clientX et event.clientY : renvoie les coordonnées de la souris pour les événement lié à la souris
- preventDefault() : empêche le comportement par défaut du navigateur pour que l’action soit entièrement gérée avec JavaScript (exemple : empêcher la soumission d’un formulaire ou la redirection d’un lien).
- StopPropagation() : empêche un événement de remonter aux éléments parents qui possèdent un addEventListener pour le même type d’événement.

Partie 4 : Allez plus loin avec JavaScript

I) La POO en Javascript

Les **classes** sont une manière moderne et lisible de créer des objets et de gérer l’**héritage** en JavaScript.

1) Définir une classe

On peut créer un modèle d’objet avec le mot-clé class. L’objet possédera les propriétés et les méthodes définies dans la classe.
Exemple :

```
class Personne {
    #age   // propriété privée
    constructor(nom, age) {
        this.nom = nom;    // propriété publique par défaut
        this.#age = age;
    }
    sePresenter() {
        console.log(`Je suis ${this.nom} et j'ai ${this.#age} ans.`);
    }
    getAge() {
        return this.#age
    }
    setAge(nouvelAge) {
        this.#age = nouvelAge
    }
}
```

```
const jean = new Personne('Jean', 30);
jean.sePresenter(); // Je suis Jean et j'ai 30 ans.
console.log(jean.nom);    // Jean
console.log(jean.getAge()); // 30
// console.log(jean.#age); // ❌ Erreur : propriété privée
```

JS ne supporte pas les attributs protégés nativement.
Les méthodes peuvent être définit comme privée avec un # devant son nom.

2) Hériter d'une classe

extends permet à une classe d'hériter des propriétés et méthodes d'une autre classe.
super( ) appelle le constructor de la classe parente.
Exemple :

```
class Employe extends Personne {
    constructor(nom, age, poste) {
        super(nom, age); // appel du constructor de Personne
        this.poste = poste;
    }
    travailler() {
        console.log(`${this.nom} travaille en tant que ${this.poste}.`);
    }
}
const karim = new Employe('Karim', 28, 'développeur');
karim.sePresenter(); // hérité de Personne
karim.travailler();  // spécifique à Employe
```

II) Communiquer avec un serveur en JavaScript : `fetch` et AJAX

`fetch` est une fonction JavaScript qui permet **d'envoyer une requête HTTP à un serveur et de récupérer sa réponse de manière asynchrone** (sans recharger la page). On parle alors de requête **AJAX**.
Les requêtes AJAX sont très pratiques pour charger ou mettre à jour des données dynamiquement, par exemple :

- liker un contenu
- Ajouter un produit dans un panier

1) Récupérer des données avec une requête AJAX GET

```
fetch('https://monsite/data')                  // 1
    .then(response => response.json())        // 2
    .then(data => console.log(data))         // 3
    .catch(error => console.error(error)); // 4
```

1. Avec fetch on contacte le serveur à l'URL https://monsite/data. Cette fonction renvoie une **promesse** qui sera résolue quand la réponse du serveur arrivera.
2. On récupère la réponse du serveur et on la convertit en objet JavaScript (on réalise la promesse)
3. L'objet obtenu est stocké dans data et peut être utilisé comme on veut. Ici on l'affiche dans la console.
4. catch permet de gérer les éventuelles erreurs, c'est à dire si la promesse n'est pas tenue)

2) Envoyer des données avec une requête AJAX POST

```
fetch('https://api.example.com/utilisateurs', {
    method: 'POST',                                  // 1
    headers: { 'Content-Type': 'application/json' }, // 2
    body: JSON.stringify({ nom: 'Alice' })          // 3
})
.then(response => response.json())        // 4
.then(data => console.log(data))         // 5
.catch(error => console.error(error));  //6
```

1. On indique à fetch que la requête se fera en POST (GET par défaut)
2. On configure l'envoi des données au format JSON via l'en-tête Content-Type
3. On transforme l'objet JS qui représente les données à envoyer au serveur en texte JSON
4. On récupère la promesse renvoyée par le serveur et on la convertit en objet JS
5. On utilise la réponse du serveur si elle existe
6. On gère les erreurs si la promesse n'est pas tenue.

3) Le format JSON

JSON (JavaScript Object Notation) est un format de texte qui permet de transmettre des données entre un serveur et un navigateur.
Il est simple, lisible et compris par presque tous les langages.
Il permet de stocker des paires clé/valeur et se note : { "nom": "Alice", "age": 25 }

On peut y stocker des informations complexe :

- des objets : { }
- des tableaux : [ ]
- des booléens ; true ou false
- des dates sous forme de texte

```
{
  "utilisateur": {
    "nom": "Alice",
    "age": 25,
    "adresse": {
      "rue": "123 rue des Fleurs",
      "ville": "Paris",
      "codePostal": "75001"
    },
    "loisirs": ["lecture", "natation", "jeux vidéo"]
  },
  "actif": true,
  "derniereConnexion": "2026-03-13T10:30:00Z"
}
```